

TRY IT YOURSELF

15-3. Molecular Motion: Modify *rw_visual.py* by replacing `ax.scatter()` with `ax.plot()`. To simulate the path of a pollen grain on the surface of a drop of water, pass in the `rw.x_values` and `rw.y_values`, and include a `linewidth` argument. Use 5,000 instead of 50,000 points to keep the plot from being too busy.

15-4. Modified Random Walks: In the `RandomWalk` class, `x_step` and `y_step` are generated from the same set of conditions. The direction is chosen randomly from the list `[1, -1]` and the distance from the list `[0, 1, 2, 3, 4]`. Modify the values in these lists to see what happens to the overall shape of your walks. Try a longer list of choices for the distance, such as 0 through 8, or remove the `-1` from the `x-` or `y-direction` list.

15-5. Refactoring: The `fill_walk()` method is lengthy. Create a new method called `get_step()` to determine the direction and distance for each step, and then calculate the step. You should end up with two calls to `get_step()` in `fill_walk()`:

```
x_step = self.get_step()
y_step = self.get_step()
```

This refactoring should reduce the size of `fill_walk()` and make the method easier to read and understand.

Rolling Dice with Plotly

In this section, we'll use Plotly to produce interactive visualizations. Plotly is particularly useful when you're creating visualizations that will be displayed in a browser, because the visualizations will scale automatically to fit the viewer's screen. These visualizations are also interactive; when the user hovers over certain elements on the screen, information about those elements is highlighted. We'll build our initial visualization in just a couple lines of code using *Plotly Express*, a subset of Plotly that focuses on generating plots with as little code as possible. Once we know our plot is correct, we'll customize the output just as we did with Matplotlib.

In this project, we'll analyze the results of rolling dice. When you roll one regular, six-sided die, you have an equal chance of rolling any of the numbers from 1 through 6. However, when you use two dice, you're more likely to roll certain numbers than others. We'll try to determine which numbers are most likely to occur by generating a dataset that represents rolling dice. Then we'll plot the results of a large number of rolls to determine which results are more likely than others.

This work helps model games involving dice, but the core ideas also apply to games that involve chance of any kind, such as card games. It also relates to many real-world situations where randomness plays a significant factor.